Studio Nx

백엔드 268 — 프로젝트 라우터 (전체 조회 · 곡 설정)

2026-10-19

Backend(FastAPI/Python) 268 — 프로젝트 라우터 (`GET /projects/{id}/full` · `PATCH /projects/{id}/settings`)

항목	내용
파일	`studionx/backend/app/api/v1/projects.py`
작업	S1-10 프로젝트 전체 저장·불러오기: 전체 조회와 곡 설정 라우터 (PRD 02 FR-253, FR-254)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	267 `update_settings` · 269 이후 화면 JS (`loadProject` · 명령 레지스트리)

1. 하는 일

P1-13 라우터(049 `p1-13_project_api`)에 두 길을 더합니다. 라우터는 얇게 두고 규칙은 서비스에 맡깁니다.

메서드 · 경로	하는 일	권한
`GET /projects/{id}/full`	프로젝트 + 트랙(+악기 · 이펙트 · 연결선) + 패턴(+노트)	멤버면 누구나
`PATCH /projects/{id}/settings`	곡 전체 설정(BPM · 박자 · Key · 스케일 · 샘플레이트 · 마스터)	OWNER · EDITOR

2. 언제 쓰나

쓰는 곳	
시퀀서를 열 때	`GET .../full` 한 번으로 화면 전체를 그립니다 (FR-254)
Ctrl+S · 자동 저장	바뀐 곡 설정만 `PATCH .../settings` 로 보냅니다 (FR-253)
곡 설정 응답	저장한 결과를 프로젝트 칸 모양으로 돌려주어 화면이 그대로 표시합니다

3. 코드

```
@router.get("/{project_id}/full", response_model=FullProjectResponse, responses=NOT_FOUND)
def read_full_project(project_id: int, user: CurrentUser, db: Db) -> FullProjectResponse:
    """프로젝트 전체 — 트랙·악기·이펙트·연결선·패턴·노트를 한 번에 (FR-254)."""
    return full_project_service.get_full_project(db, user, project_id)


@router.patch(
    "/{project_id}/settings",
    response_model=FullProjectInfo,
    responses={**NOT_FOUND, **FORBIDDEN},
)
def update_project_settings(
    project_id: int,
    data: FullProjectSettingsUpdateRequest,
    user: CurrentUser,
    db: Db,
) -> FullProjectInfo:
    """곡 전체 설정 저장 (FR-253) — OWNER · EDITOR."""
```

```
    project, member = project_service.get_membership(db, project_id, user)
    project = project_service.update_settings(db, project, member, data)
    return full_project_service.project_info(db, project, member)
```

문법 풀이

식	뜻
`CurrentUser` · `Db`	`app/api/deps.py` 별칭입니다. 로그인한 사용자와 DB 세션을 자동으로 받습니다
`"/{project_id}/full"`	경로 뒤에 `full` 을 붙입니다. `GET /projects/{id}` (069) 와 겹치지 않습니다
`response_model=FullProjectInfo`	곡 설정 저장 뒤에도 프로젝트 칸 전체를 돌려줍니다
`responses={**NOT_FOUND, **FORBIDDEN}`	문서에 404 · 403 모양을 적어 둡니다
`**` 두 사전 합치기	`NOT_FOUND`(404) 와 `FORBIDDEN`(403) 을 한 사전으로 만듭니다

4. 왜 이렇게

선택	이유
라우터는 얇게	권한 · 404 규칙은 서비스 한 곳에 모읍니다. 라우터는 부르기만 합니다
전체 조회를 `/projects` 아래에 둠	프로젝트 하나의 상태라 프로젝트 자원 아래가 맞습니다. 트랙 · 패턴 라우터는 그대로 둡니다
곡 설정을 `settings` 경로로	제목 · 설명(`PATCH /projects/{id}`)과 섞지 않습니다. 시퀀서가 곡 설정만 보내기 쉽습니다
401 은 자동	`CurrentUser` 가 로그인을 요구합니다
응답이 프로젝트 칸	저장 직후 화면이 값을 다시 묻지 않아도 됩니다

5. 시험

`tests/test_full_project_routes.py` 의 `test_full_route_registered` 와 `tests/test_project_settings_routes.py` 의 `test_settings_route_registered` 가 `app.openapi()["paths"]` 에 두 경로가 실렸는지 확인합니다. 같은 파일들의 나머지 시험은 실제 앱 · MySQL 로 요청해 응답과 권한(200 · 401 · 403 · 404 · 422)을 확인하며 모두 통과합니다.